

Kitchen Bots Backend Status and Implementation Report

Prepared for: Project leadership

Report date: 21 September 2026

Applications reviewed: Kitchen Bots e-commerce storefront and operations dashboard

Executive summary

The project has a functional storefront interface, a separately structured dashboard application, tested frontend service abstractions, and a legacy Google Apps Script integration layer. It does not yet have a production-ready backend.

The current dashboard can demonstrate workflows and interface behavior, but authentication, authorization, product management, inventory, reporting, and several operational datasets are still mocked or handled in the browser. Google Apps Script and Google Sheets provide an existing CRUD path for selected modules, but this layer does not provide the security, validation, concurrency controls, auditability, or maintainability required for production commerce operations.

The recommended production architecture is:

- Firebase Authentication for staff and customer identity
- Firestore for structured operational data
- Cloudflare Workers with Hono for the public and protected API
- Cloudflare R2 for product media and private documents
- Cloudflare Turnstile for public form abuse protection
- Resend, or an equivalent transactional email provider, for notifications
- The existing React storefront and dashboard as separate deployable applications

Payments should remain deferred until the client selects a payment provider and confirms merchant-account requirements.

1. Assessment method

This report distinguishes capabilities using the following definitions:

- **Verified Working:** Code exists and the relevant local build or automated tests passed.
- **Partially Implemented:** Interfaces or integration code exist, but production behavior was not fully verified or still depends on mocks or legacy infrastructure.
- **Not Implemented:** The required production capability is absent.

The live Google Apps Script deployment, Firebase services, Cloudflare infrastructure, and R2 contents were not modified or live-tested during this assessment.

2. Verified working capabilities

Dashboard application foundation

- Separate React dashboard application
- Admin and customer route structures
- Protected-route and guest-route components
- React Router navigation
- TanStack Query integration
- Dashboard layouts and reusable interface components
- Role and permission utility logic
- Product and inventory service interfaces
- Local event bus
- Local order and quotation transition logic

The dashboard production build completed successfully. Its automated suite passed 55 tests across 11 test files during the assessment.

Storefront integration boundary

- The storefront and dashboard are separated into independent applications.
- Storefront account access can be directed to the dashboard using `VITE_PORTAL_URL`.
- The storefront cart now routes customers toward an enquiry workflow instead of presenting a simulated payment checkout.
- CI checks cover type checking, linting, tests, and the production build.

Legacy Google Apps Script layer

The repository contains a Google Apps Script and Google Sheets CRUD layer for:

- Products
- Orders
- Leads
- Users
- Documents
- Services
- Activity logs

The dashboard contains a configured Google Apps Script URL variable and client modules that send JSON requests to this service.

This confirms that an integration path exists. It does not confirm that the deployed endpoint is currently available, secure, or production-ready.

Media integration foundation

- R2-oriented product media repository code exists.
- Media URL resolution code exists in the storefront.

- Product-media upload tooling exists.

The current live R2 inventory and CDN behavior were not verified during this assessment.

3. Partially implemented capabilities

Capability	Current condition	Production gap
Authentication	Mock service and login interfaces exist	No Firebase Authentication integration or trusted server session
Authorization	Permission utilities and protected UI routes exist	Roles can be influenced client-side; no server enforcement
Products	Local services and Apps Script API client exist	No canonical Firestore model, publishing workflow, or secure API
Inventory	In-memory inventory service exists	No persistent stock ledger, reservations, or concurrency handling
Orders	UI and transition logic exist	No authoritative server-side order creation or total calculation
Enquiries and leads	Interfaces and legacy modules exist	No protected production endpoint, validation, spam control, or ownership workflow
Documents	Legacy module exists	No private R2 access model or signed document URLs
Reporting	Dashboard screens exist	Most statistics and activity data are mocked
Content management	Local content service exists	No persistent CMS workflow or publishing controls
Email	UI concepts may reference notifications	No transactional mail provider, outbox, retry, or delivery tracking

4. Mocked or unsafe current behavior

- Authentication uses development credentials and mock tokens.
- Session and role-switching state is stored in the browser.
- Dashboard revenue, statistics, orders, leads, activities, and tickets include mock arrays.
- Product and inventory services use in-memory data and simulated delays.
- Content, notifications, permissions, and activity services use local data.
- Some filtering and pagination are performed by the browser.
- The current-user API behavior can select the first returned user rather than resolving a verified identity.
- Google Apps Script routes do not provide the required production authentication and authorization boundary.
- There is no complete schema validation layer at the API boundary.
- There is no documented rate limiting, request idempotency, or Turnstile enforcement.
- There is no production-grade stock concurrency control.
- There is no verified immutable audit trail for sensitive changes.

These components are useful for interface development and demonstrations, but they must not be treated as production backend functionality.

5. Production capabilities not yet implemented

Identity and access

- Firebase Authentication integration
- Email/password and Google sign-in
- Secure password recovery
- Server-managed role claims
- Staff invitation and role-management workflow
- Organization membership and tenant isolation
- Server-side authorization for every protected operation

Catalog and CMS

- Canonical product and category schemas
- Product publishing states
- Specifications, descriptions, pricing, availability, and media associations
- Dashboard product editor connected to persistent data
- Content blocks for editable storefront pages
- Data migration from hardcoded frontend product files

Enquiries, quotations, and orders

- Public enquiry API
- Turnstile validation and rate limiting
- Enquiry assignment and claiming
- Quotation creation and revision
- Server-side order creation
- Server-side price, tax, and availability calculation
- Order status workflow
- Idempotency protection
- Customer-visible order history

Inventory

- Persistent inventory records
- Stock adjustments and reasons
- Reservation and release behavior
- Low-stock thresholds
- Concurrency-safe updates
- Inventory audit history

Files and notifications

- Public and private R2 bucket policies
- Signed access for private documents
- Secure upload validation
- Transactional email provider
- Email outbox, retries, and delivery status

Platform operations

- Development and production Firebase projects
- Firestore security rules and indexes
- Firestore emulator tests
- Cloudflare Worker deployment
- Environment and secret management
- Structured logs and request identifiers
- Error monitoring and alerting
- Backup and recovery procedures
- Deployment runbooks
- Data-retention policy

Payments

- Payment-provider selection
- Merchant-account configuration
- Server-created payment sessions
- Signed webhook verification
- Idempotent payment processing
- Refund and reconciliation workflows

Payment implementation should start only after the provider, settlement process, refund policy, and compliance responsibilities are confirmed.

6. Recommended target architecture

Client applications

- Storefront: public catalog, product details, enquiries, customer account entry point
- Dashboard: staff operations, CMS, CRM, orders, inventory, service requests, and customer portal

API boundary

A Hono application deployed on Cloudflare Workers should become the only trusted API boundary. It should validate request payloads, verify Firebase identity tokens, enforce roles, recalculate commercial values, and write audit events.

Initial API surface:

- GET /v1/catalog/products
- GET /v1/catalog/products/:slug
- POST /v1/enquiries
- POST /v1/orders
- POST /v1/enquiries/:id/claim
- GET /v1/documents/:id/access
- POST /v1/admin/staff-claims

Primary data collections

- users
- organizations
- memberships
- products
- categories
- content
- enquiries
- quotes
- orders
- inventoryEvents
- serviceRequests
- documents
- auditEvents
- mailOutbox
- idempotencyKeys
- payments
- paymentEvents

Payment collections can remain inactive until the payment phase is approved.

7. Recommended delivery plan

Phase 1: Contracts and security foundation

1. Finalize domain schemas and API contracts.
2. Represent monetary values as integer paise.
3. Create separate Firebase development and production projects.
4. Implement Firebase Authentication and trusted role claims.
5. Create Firestore collections, indexes, and deny-by-default rules.
6. Add emulator tests for access rules.
7. Establish the Hono Worker with validation, authentication middleware, CORS, request IDs, and consistent errors.

Phase 2: Catalog and enquiry slice

1. Move products and categories into Firestore.
2. Connect public product media to R2/CDN URLs.
3. Implement published-only catalog endpoints.
4. Connect storefront search and product details to the API.
5. Implement enquiry submission with validation, Turnstile, rate limiting, and idempotency.
6. Connect dashboard enquiry management.

Phase 3: Dashboard operations

1. Replace mock and Google Apps Script services incrementally.
2. Implement product and content publishing workflows.
3. Implement quotation and order workflows.
4. Implement inventory adjustments and audit records.
5. Add private document access and transactional email.

Phase 4: Customer portal and migration

1. Restrict customer access to their own records.
2. Add order, quotation, service, and document views.
3. Migrate required Google Sheets records.
4. Retire the Google Apps Script layer after reconciliation.
5. Complete deployment, monitoring, backup, and operational runbooks.

Phase 5: Payments

Implement payments only after client approval and provider selection. Payment totals must be calculated by the server, and webhook processing must be signed and idempotent.

8. Principal risks

Risk	Impact	Required control
Mock authentication mistaken for production auth	Unauthorized access	Replace with Firebase Auth and server claims before production use
Client-side pricing or role decisions	Data and financial manipulation	Enforce all sensitive decisions in the Worker
Google Sheets used as primary production database	Concurrency, validation, and audit failures	Migrate canonical data to Firestore
Public access to private documents	Data exposure	Separate R2 access policies and signed URLs
Duplicate enquiry or order submissions	Duplicate operational records	Idempotency keys and request-state tracking
Hardcoded catalog data	Stale and inconsistent product information	CMS and catalog migration

Risk	Impact	Required control
Mock dashboard metrics	Incorrect management decisions	Replace with queries over canonical records

9. Immediate decisions required from project leadership

- Confirm the authoritative product catalog and required product fields.
- Confirm staff roles and approval permissions.
- Confirm the enquiry-to-quotation-to-order workflow.
- Confirm inventory ownership and adjustment rules.
- Confirm which customer portal functions are required for the first release.
- Confirm email sender domain and notification requirements.
- Confirm whether legacy Google Sheets data must be migrated.
- Confirm payment-provider requirements and whether payments are part of the initial release.
- Provide approved legal, policy, company, and contact information for production content.

10. Current conclusion

The project has a usable frontend foundation and sufficient service abstractions to begin backend implementation without rebuilding the interfaces. The immediate priority is not adding more dashboard screens. It is establishing the trusted backend boundary, production identity, canonical data model, and one complete catalog-to-enquiry workflow.

The system should not be presented as production-ready until authentication, authorization, persistent data, API validation, audit logging, deployment controls, and security tests are implemented and verified.